

Data Structures and Algorithms

What is data?

Data

- A collection of facts from which conclusion may be drawn
- e.g. Data: Temperature 35°C; Conclusion: It is hot.

Types of data

- Textual: For example, your name (Rajesh)
- Numeric: For example, your ID (090254)
- Audio: For example, your voice
- Video: For example, your voice and picture
-

What is data structure?

- A particular way of storing and organizing data in a computer so that it can be used efficiently and effectively.
- Data structure is the logical or mathematical model of a particular organization of data.
- A bunch of data elements grouped together under one name.
 - For example, an array of integers

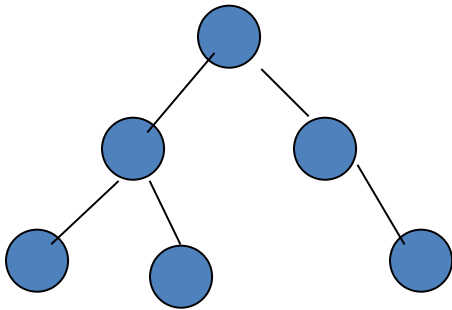
Types of data structures



Array



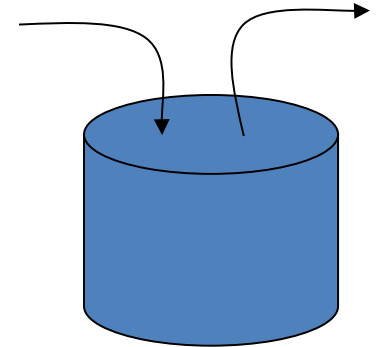
Linked List



Tree



Queue



Stack

There are many, but we named a few. We'll learn these data structures in great detail!

Classification of Data Structures

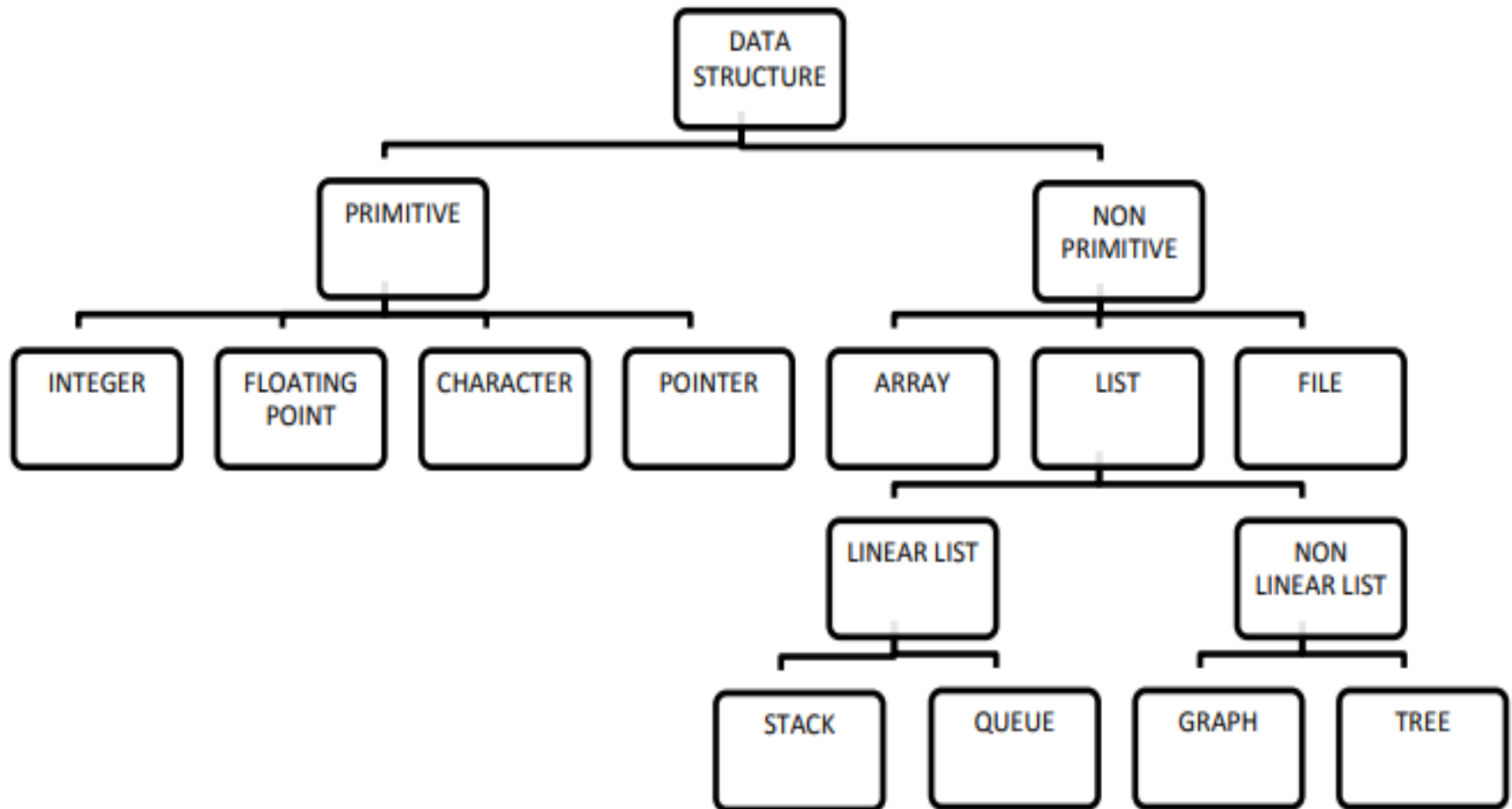
Primitive data structures:

- They are the basic data structures that are available in most of the programming languages.
- They are used to represent single value.

Non-Primitive data structures:

- They are derived from primitive data structures.
- They are used to store a group of values.

Classification of Data Structures



The Need for Data Structures

- **Goal:** to organize data
- **Criteria:** to facilitate **efficient**
 - **storage** of data
 - **retrieval** of data
 - **manipulation** of data
- **Examples:**
 - Telephone Directory
 - What if it is not sorted?
 - Queue at Box office for movie ticket
 - What if it is not necessary to maintain queue?

Data Structure Operations

- Traversing

- It is used to access each data item exactly once so that it can be processed.

- Searching

- It is used to find out the location of the data item if it exists in the given collection of data items.

- Insertion

- It is used to add a new data item in the given collection of data items.

Data Structure Operations (cont.)

- Deletion

- It is used to delete an existing data item from the given collection of data items.

- Sorting

- It is used to arrange the data items in some order i.e. in ascending or descending order in case of numerical data and in dictionary order in case of alphanumeric data.

- Merging

- It is used to combine the data items of two sorted files into single file in the sorted form.

What is an algorithm?

- An algorithm is a finite set of instructions or logic, written in order, to accomplish a certain predefined task.
- Algorithm is not the complete code or program, it is just the core logic(solution) of a problem
- can be expressed either as an informal high level description as **pseudo code** or using a **flowchart**.

Characteristics of an Algorithm

- **Input-** There should be 0 or more inputs supplied externally to the algorithm.
- **Output-** There should be at least 1 output obtained.
- **Definiteness-** Every step of the algorithm should be clear (Unambiguous) and well defined.
- **Finiteness-** The algorithm should have finite number of steps.
- **Correctness-** Every step of the algorithm must generate a correct output.

How to express an algorithm?

- A sequence of steps to solve a problem
- We need a way to express this sequence of steps
 - **Natural language (NL)** is an obvious choice, but not a good choice. Why?
 - NLs are notoriously ambiguous (unclear)
 - **Programming language (PL)** is another choice, but again not a good choice. Why?
 - Algorithm should be PL independent
- We need some balance
 - We need PL independence
 - We need clarity
 - Pseudo-code provides the right balance

What is pseudo-code?

- Pseudo-code is a short hand way of describing a computer program
- Rather than using the specific syntax of a computer language, more general wording is used
- It is a mixture of NL and PL expressions, in a systematic way
- Using pseudo-code, it is easier for a non-programmer to understand the general workings of the program

Pseudo-code: general guidelines

- Use PLs construct that are consistent with modern high level languages, e.g. C, C++, Java, ...
- Use appropriate comments for clarity
- Be simple and precise

Components of Pseudo-code

▪ Expressions

- Standard mathematical symbols are used
 - Left arrow sign ($\leftarrow$) as the assignment operator in assignment statements (equivalent to the `=` operator in C)
 - Equal sign (`=`) as the equality relation in Boolean expressions (equivalent to the `"= ="` relation in C)
 - For example
 - `Sum  $\leftarrow$  5`
 - `if(Sum = 6)`

Components of Pseudo-code (cont.)

- **Decision structures** (if-then-else logic)
 - **if** condition **then** true-actions [**else** false-actions]
 - We use indentation to indicate what actions should be included in the true-actions and false-actions
 - For example

```
if marks > 50 then
    print "Congratulation, you are passed!"
else
    print "Sorry, you are failed!"
end if
```


Components of Pseudo-code (cont.)

- Loops (Repetition)

- **Pre-condition loops**

- While loops
 - **while** condition **do** actions
 - We use indentation to indicate what actions should be included in the loop actions
 - For example

- while** counter < 5 **do**

- print “Welcome to the Course!”

- counter ← counter + 1

- end while**

Components of Pseudo-code (cont.)

- Loops (Repetition)

- **Pre-condition loops**

- For loops

- **for** variable-increment-definition **do** actions

- For example

- for** counter $\leftarrow$ 0; counter $<$ 5; counter $\leftarrow$ counter + 2
 - do**

- print “Welcome to the Course!”

- end for**

What will be the output?

Components of Pseudo-code (cont.)

- Loops (Repetition)

- **Post-condition loops**

- Do loops
 - **do** actions **while** condition
 - For example
counter $\leftarrow$ 1
do
 print “Welcome to the Course!”
 counter $\leftarrow$ counter + 1
while counter < 5

The body of a post-condition loop must execute at least once

Components of Pseudo-code (cont.)

▪ **Method declarations**

- Return_type method_name (parameter_list) method_body

- For example

```
integer sum ( integer num1, integer num2)
start
    result ← num1 + num2
    return result
end
```

▪ **Method calls**

- result ← method (args)

- For example

```
total ← sum(num1, num2)
```

Components of Pseudo-code (cont.)

■ Comments

- `/*` Multiple line comments go here. `*/`
- `//` Single line comments go here
- Some people prefer braces `{}`, for comments

■ Arrays

- $A[i]$ represents the i th cell in the array A .
- The cells of an n -celled array A are indexed from $A[1]$ to $A[n]$.

Importance of Data Structures and Algorithms

Programs = Data Structures + Algorithms

-- Niklaus Wirth, author of Pascal language

From above we can say that

Efficient Programs = Efficient (Data Structures + Algorithms)

Efficient Programs = Efficient Data Structures (This Course)

+

Efficient Algorithms (Next Semester)

Homework

1. Write pseudo-code to display first N odd/even numbers.
2. Write an algorithm in pseudo code that finds the average of (n) numbers.